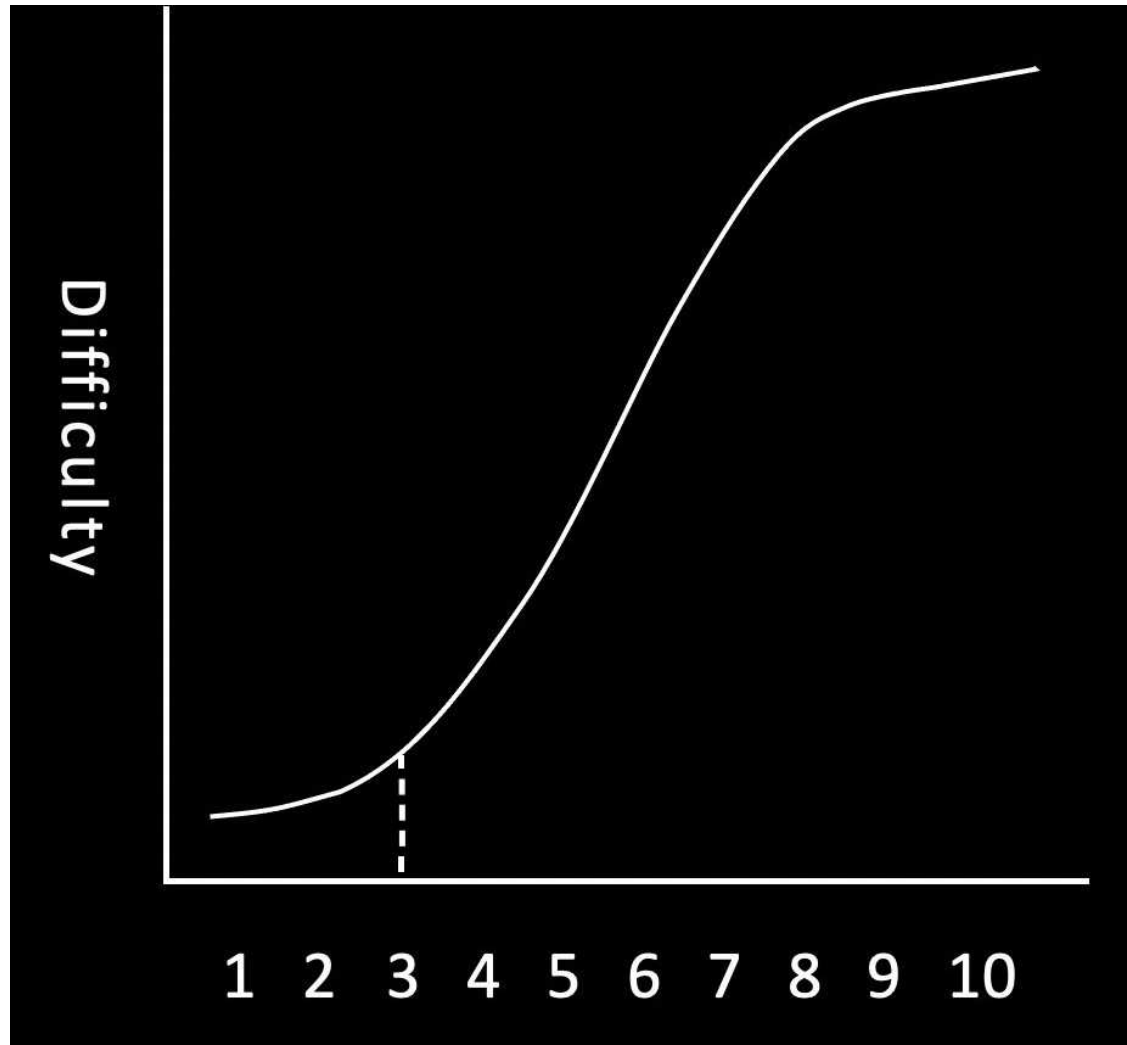


# COMS30020 - Computer Graphics

## Week 3 Briefing

Dr Simon Lock

# Where are we ?



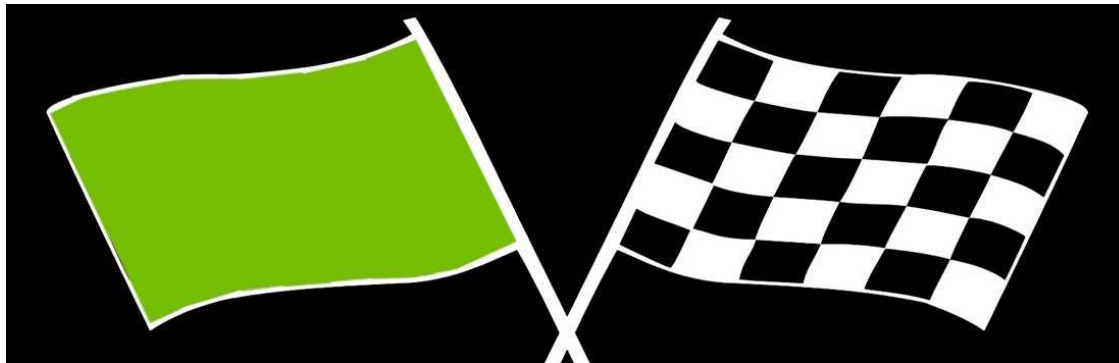
# This week on "Computer Graphics"

We start doing some serious structured drawing !

Our focus will be on a key drawing primitive:

△ △ △ "The Triangle" △ △ △

The primary building block for the rest of this unit !!!



# Powerful Things

Although simple, Triangles are VERY powerful  
A convenient structure to cover ANY surface



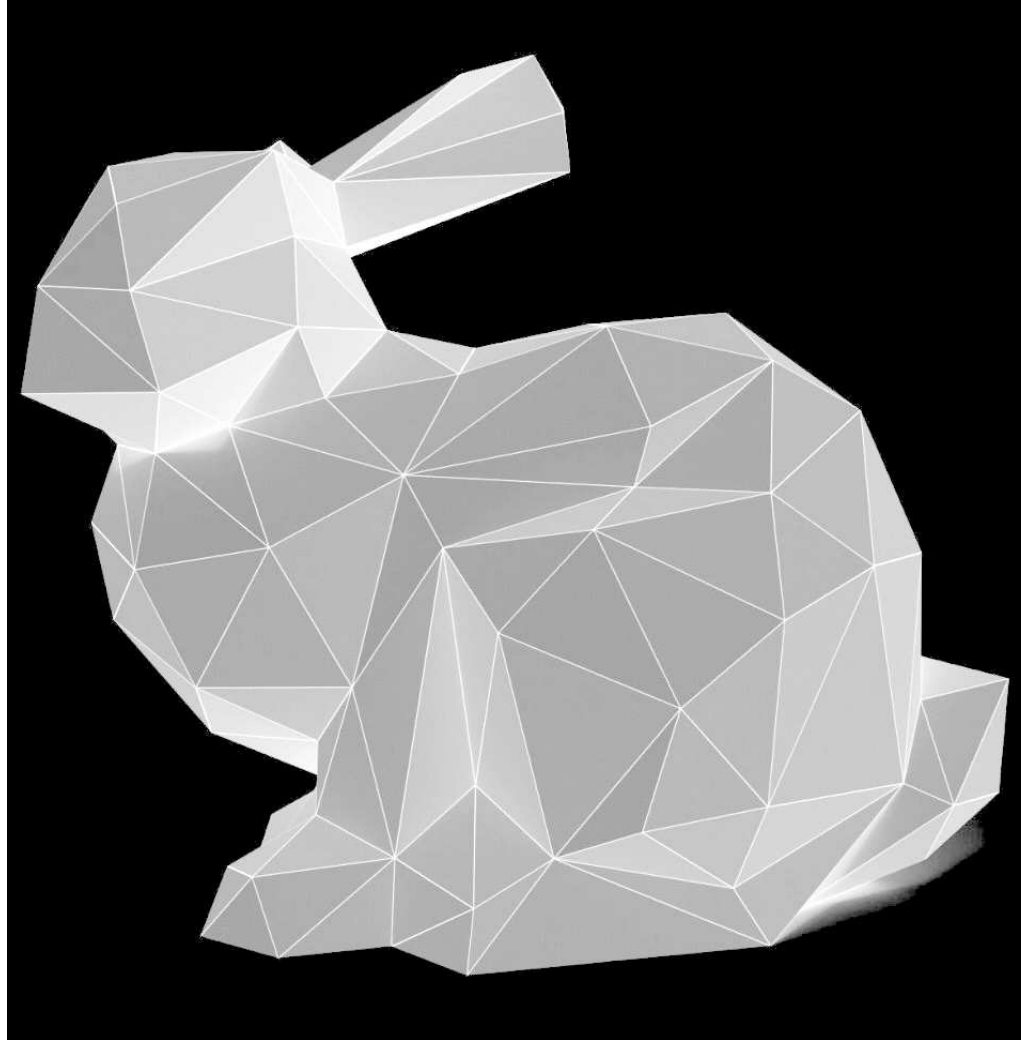


With MANY triangles,  
We can create VERY complex shapes...

# The Stanford Bunny (70k triangles)



# Low Poly Equivalent



# What kinds of Triangle ?

We will actually be drawing various types of triangle:

- Unfilled (also known as "stroked") triangles
- Filled triangles (with a choice of colours !)
- Composite triangles (filled AND stroked !!!)

## THE CHALLENGE

We won't be using an existing `drawTriangle` function

We will instead write our own triangle drawing code

This is however non-trivial (the devil is in the detail)

There is hidden complexity that we must deal with !

# Stroked Triangles

Surely a "stroked" triangle is easy ?

After all, it is just three straight lines !



BUT currently we don't have a `drawLine` function

We could just use the `setPixelColour` function...

Call it lots of times to draw a sequence of pixels

We just need to calculate X and Y

for each point along the line

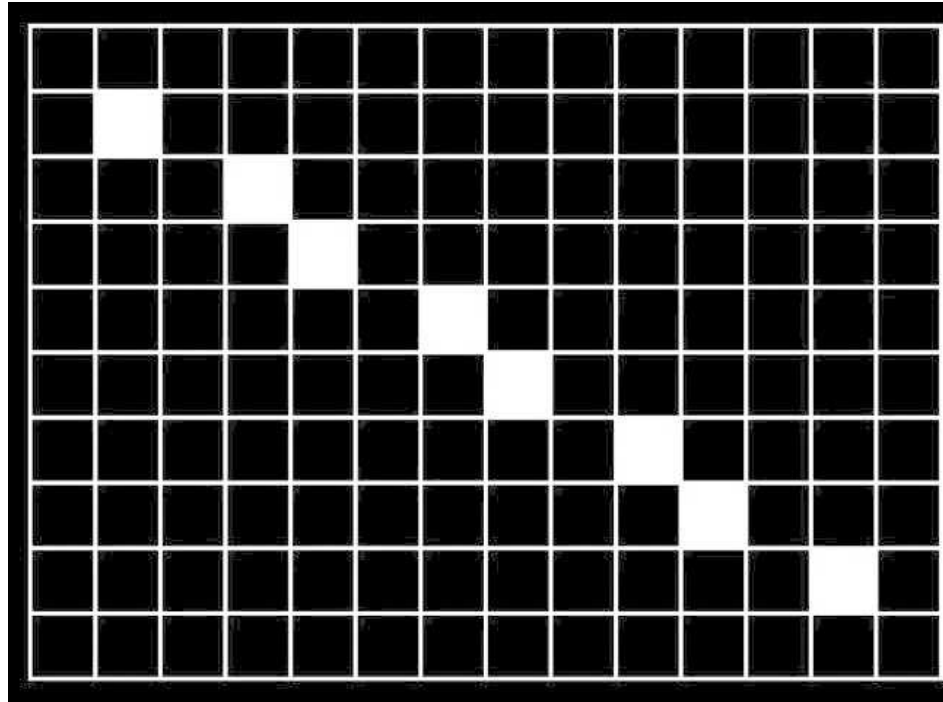
(using a nice bit of interpolation !)

# Not that easy !

If we just loop through each ROW like this:

```
for(int y=1; y<=8 ;y++) {
```

We could end up drawing something like this:

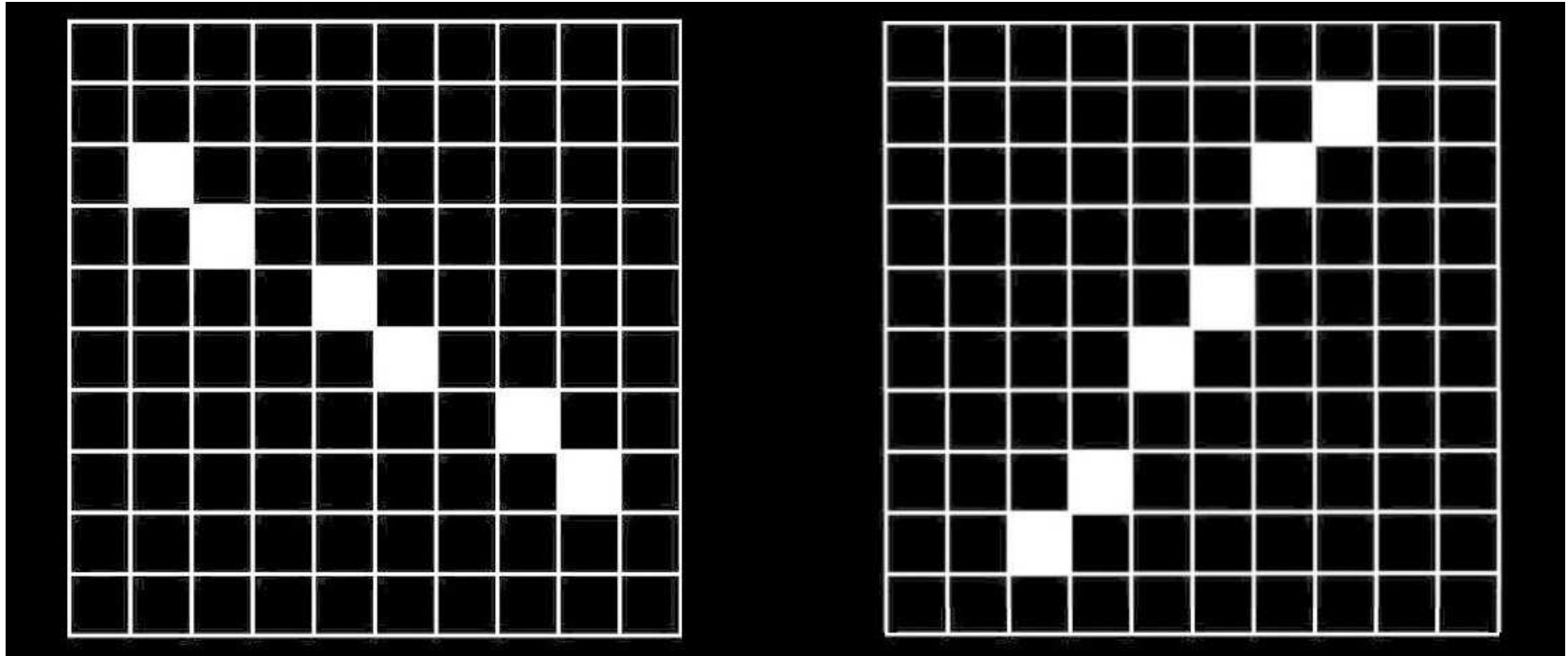


# Needs Careful Handling

Neither can we ALWAYS just loop through columns !

We must consider gradient of line when drawing pixels

The workbook provides details on how to deal with this



# Filled Triangles - Are they any easier ?

Filled triangles must surely be easier then ?

All we need to do is draw a stack of horizontal lines  
Starting and ending at the correct x positions

For each horizontal line:

We just draw from "the left-hand side"...

All the way to "the right-hand side"

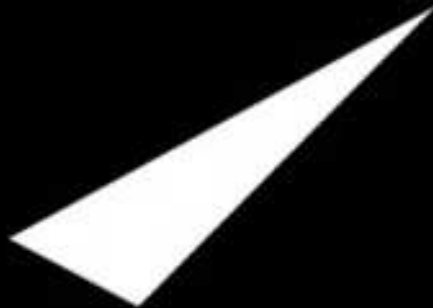
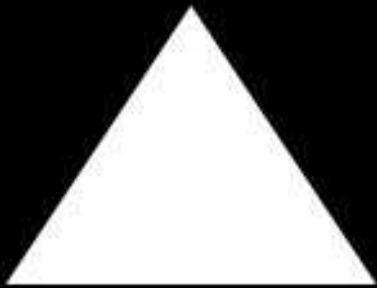
One row at a time

Easy !

(?)

# The Problem: Different TYPES of triangle

The first triangle below ("flat bottomed") is easy !  
However, non flat top/bottom triangles are harder  
There isn't a "left-hand side" and "right-hand side"  
Because each triangle has THREE sides...

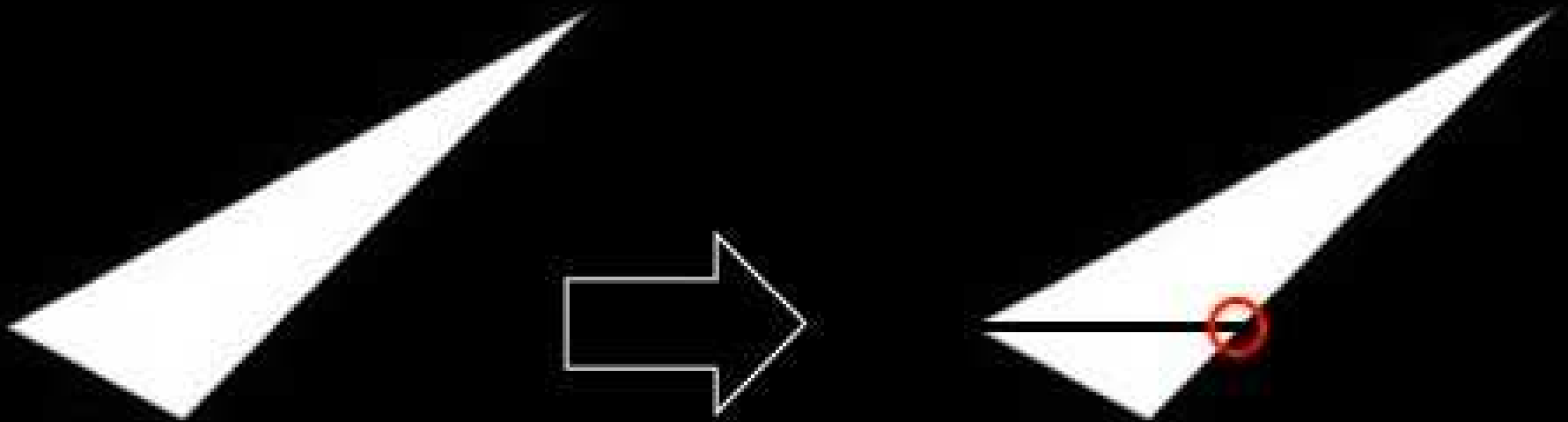


# How to cope with these ?

Workbook explains a technique for dealing with these !

We basically split each triangle into 2 "easier" ones

To achieve this, we need a 4th vertex - an "Extra Point"



And if all that wasn't exciting enough...

# Sergio Odeith



# Sergio Odeith



# Tom Bragado Blanco



# Tom Bragado Blanco

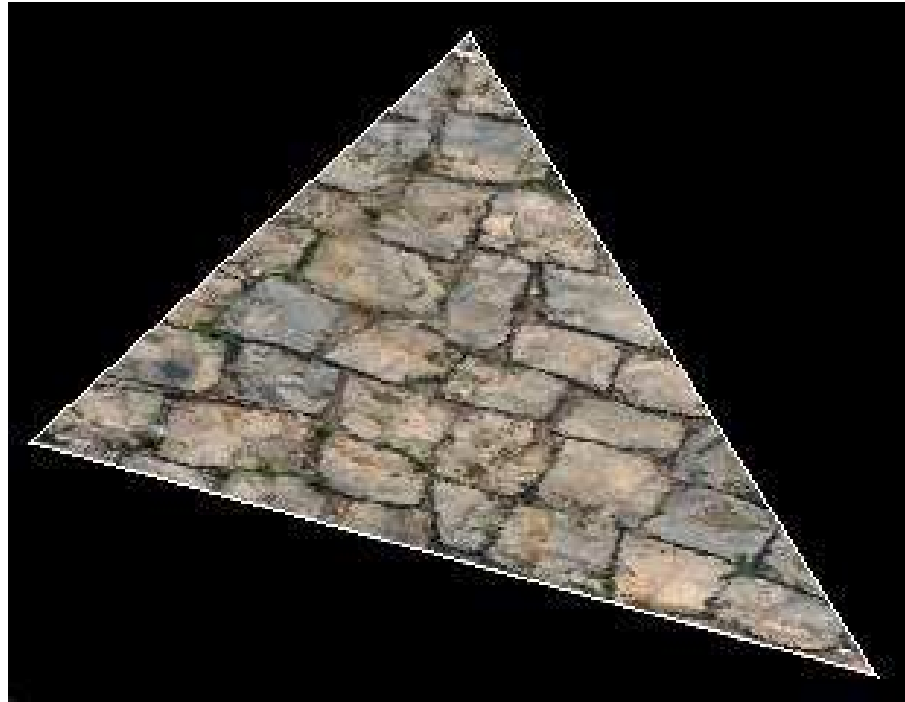


# Your Objective: a simpler texturing task

A texture is provided for you in the workbook

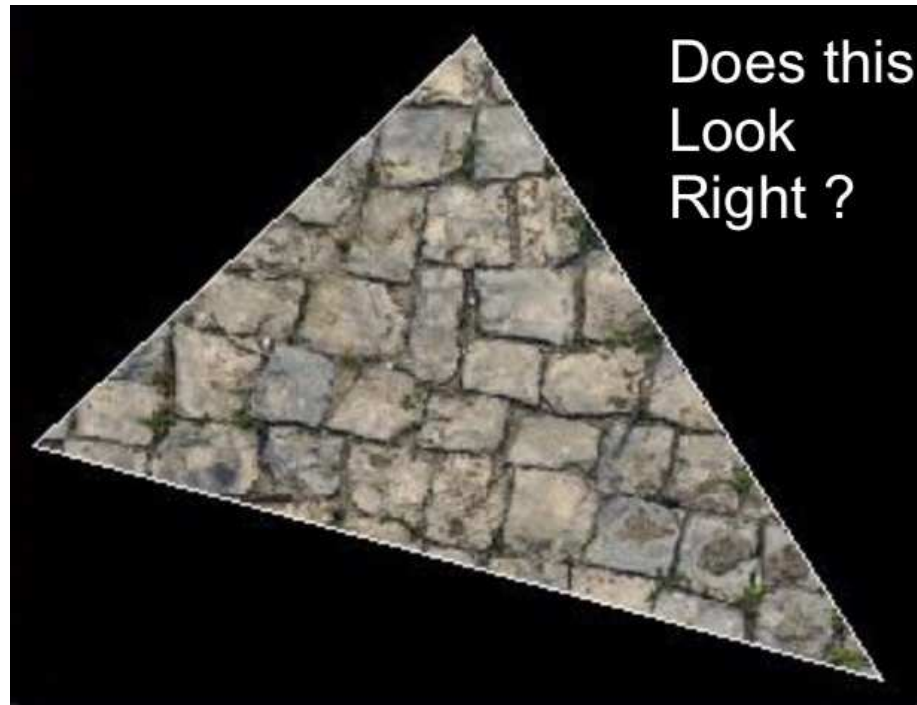
A reference image illustrates your final objective

This allows you to "visually verify" your success



# Implicit Feedback

Reference image provides passive/implicit feedback  
You can gauge how well you are currently doing  
and what aspects of your work need improving



# Approach to Texture Mapping

Texture mapping is a non-trivial activity  
(as with many things in Computer Graphics !)  
There is some hidden complexity to deal with  
Let's explore an animation for more detail...

TextureMapper

# A Festival of Interpolation !

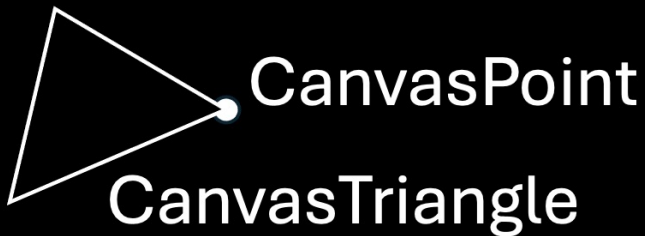
# "Key Concepts" and "Provided Classes"

All class source files can be found in `libs/sdw`

## *-Screen-*

DrawingWindow

### *-Canvas-*



## *-File System-*

*-PPM Image File-*

TextureMap

TexturePoint



# Top Tips !

You are about to embark on some complex coding

Just because it is "graphical" and "mathematical"  
You shouldn't forget GOOD programming practice

DON'T write everything as one monolithic function  
Remember to "Divide and Conquer"

Try to write modular, reusable functions  
Things you write once, then call in multiple locations

# Meaningful Names

Mathematicians use very short names  $[x,y,z]$

This is fine for tightly constrained problems

Complex and extensive code is VERY different

We soon get into difficulty if ALL names are short

Choose useful and meaningful names for everything

Makes everyone's life much easier when debugging

```

#include <stdio.h> // ./card > mattz.ppm # see https://goo.gl/JM9c2P
typedef double f;f H=.5,Y=.66,S=-1,I,y=-111;extern"C"{f cos(f),pow(f
,f),atan2(f,f);}struct v{f x,y,z;v(f a=0,f b=0,f c=0):x(a),y(b),z(c)
{}}f operator%(v r){return x*r.x+y*r.y+z*r.z;}v operator+(v r){return
v(x+r.x,y+r.y,z+r.z);}v operator*(f s){return v(x*s,y*s,z*s);}W(1,1
,1),P,C,M;f U(f a){return a<0?0:a>1?1:a;}v _(v t){return t*pow(t%t,-
H);}f Q(v c){M=P+c*S;f d=M%M;return d<I?C=c,I=d:0;}f D(v p){I=99;P=p
;f l,u,t;v k;for(const char*b="BCJB@bJBHbJCE[FLL_A[FLMCA[CCTT`T";*b;
++b){k.x+=*b/4&15;int o=*b&3,a=*++b&7;k.y=*b/8&7;v d(o%2*a,o/2*a);!o
?l=a/4%2*-3.14,u=a/2%2*3.14,d=p+k*-H,t=atan2(d.y,d.x),t=t<l?l:t>u?u:
t,Q(k*H+v(cos(t),cos(t-1.57))*(a%2*H+1)):Q(k+d*U((p+k*S)%d/(d%d)));}
return M=Q(v(p.x,-.9,p.z))?(int(p.x+64)^int(p.z+64))/8&1?Y:W:W(Y,Y,1
),pow(I,H)-.45;}v R(v o,v d,f z){for(f u=0,l=1,i=0,a=1;u<97;u+=l=D(o
+d*u))if(l<.01){v p=M,n=_(P+C*S),L=_(v(S,1,2));for(o=o+d*u;++i<6;a-=
U(i/3-D(o+n*i*.3))/pow(2,i));p=p*(U(n%L)*H*Y+Y)*a;p=z?p*Y+R(o+n*.1,d
+n*-.2*(d%n),z-1)*H*Y:p;u=pow(U(n%_(L+d*S)),40);return p+p*-u+W*u;}z=
d.z*d.z;return v(z,z,1);} int main(){for(puts("P6 600 220 255");++y<
110;)for(f x=-301;P=R(v(-2,4,25),_(_(v(5,0,2))*++x+_(v(-2,73))*-y+v(
301,-59,-735)),2)*255,x<300;putchar(P.z))putchar(P.x),putchar(P.y);}

```

